1 Лекция 06. Sunflower lemma и полиномиальное ядро d-Hitting Set

1.1 Как много похожих требований превращается в одно

В **Hitting Set** мы выбираем элементы вселенной, чтобы пересечь каждое заданное множество. Например, если требования — $\{a, b\}, \{a, c\}, \{a, d\}$, то можно взять один элемент a либо три элемента b, c, d. При бюджете 2 второй вариант невозможен. Это позволяет заменить три требования одним: выбрать a.

Мы хотим найти такую структуру в произвольной большой семье и доказать, что после исчерпания замен требований останется полиномиально мало. Множества, имеющие одинаковую общую часть и непересекающиеся остатки, называются подсолнухом. Ниже сначала точно определим эту структуру, затем докажем комбинаторную лемму о её существовании, превратим доказательство в алгоритм и только после этого получим ядро.

Здесь «сердцевина подсолнуха» и «ядро параметризованной задачи» — разные объекты. Сердцевина — пересечение нескольких множеств; ядро — весь уменьшенный экземпляр, включая семейство, вселенную и бюджет. Символ d ограничивает размер каждого требования, а k — число выбираемых элементов. Смешение этих двух величин приводит к неверным оценкам.

1.2 Постановка и комбинаторная структура

В d-Hitting Set задано конечное множество U, семейство $\mathcal{F}$ его подмножеств размера не больше d и бюджет k. Нужно $X \subseteq U$, $|X| \le k$, пересекающее каждое $F \in \mathcal{F}$. Здесь $d \ge 1$ — фиксированная константа. Дубликаты удаляем. Пустое множество во входе означает NO; пустое семейство означает YES при $k \ge 0$. Случай $k = 0$ обрабатываем до основных правил.

Подсолнух (sunflower) из r различных множеств $F_1, \dots, F_r$ имеет сердцевину C, если $F_i \cap F_j = C$ для всех $i \ne j$. Множества $F_i \setminus C$ — попарно непересекающиеся лепестки. В рассматриваемой однородной семье они непусты при $r \ge 2$.

Если $r = k + 1$, любое hitting set размера не больше k обязано пересечь C. Иначе оно должно выбрать хотя бы по одной различной вершине каждого из $k + 1$ лепестков. Это «много независимых требований» превращает локальную структуру в безопасное правило.

1.3 Лемма Эрдёша–Радо

Пусть все множества семейства имеют ровно d элементов и

$$|\mathcal{F}| > d!(r-1)^d, \qquad r \ge 2.$$

Тогда в семействе есть подсолнух из r множеств. Докажем по d.

При $d = 1$ имеем больше $r - 1$ различных одноэлементных множеств; любые r образуют подсолнух с пустой сердцевиной. Для $d > 1$ найдём жадно максимальное по включению попарно непересекающееся подсемейство $A_1, \dots, A_s$. Если $s \ge r$, ответ уже найден. Иначе $W = \bigcup A_i$ имеет размер не больше $d(r-1)$ и пересекает каждый элемент $\mathcal{F}$ по максимальности выбора.

Следовательно, некоторый $u \in W$ встречается более чем в

$$\frac{d!(r-1)^d}{d(r-1)} = (d-1)!(r-1)^{d-1}$$

множествах. Оставим только содержащие u множества и удалим из каждого u. Они остаются различными и имеют размер $d-1$. По индукции есть подсолнух из r множеств; возвращение u добавляет его к общей сердцевине и сохраняет равенство попарных пересечений.

Алгоритм следует доказательству: на каждом уровне либо готовый подсолнух, либо один рекурсивный вызов на семье меньшего ранга. Это полиномиальное время; перебирать все наборы из $k+1$ множеств не требуется.

1.4 Безопасное сжатие

Заменяем найденные $k+1$ множества подсолнуха одним множеством C, оставляя остальные требования. Вперёд: старое hitting set обязано пересечь C. Назад: пересечение C пересекает каждое удалённое множество. Удаляем элементы вселенной, больше нигде не встречающиеся. Если $C = \varnothing$, получаем NO.

Применяем правило, когда для некоторого $j \in \{1, \dots, d\}$ число множеств размера j превышает $j!k^j$. Лемма гарантирует подсолнух в этом однородном слое. При $k \geq 1$ замена уменьшает число множеств хотя бы на один, поэтому итераций не больше исходного $|\mathcal{F}|$. В конце

$$|\mathcal{F}'| \leq \sum_{j=1}^{d} j!k^j \leq d\, d!k^d, \qquad |U'| \leq d|\mathcal{F}'| \leq d^2 d!k^d.$$

Это $O(k^d)$ множеств и элементов для фиксированного d, после перенумерации также полиномиальная битовая кодировка. Используем явно эту оценку, получаемую послойным доказательством: в формулировке теоремы 2.26 и последующем доказательстве авторской PDF различается множитель d; для нашего результата он константен, но численно мы его не теряем.

Если d — часть параметра, полученная функция зависит и от d. Выражение k^d не является полиномом фиксированной степени по совокупному параметру $k+d$.

1.5 Как выглядит последовательная обработка требований

Пусть $k = 2$, $d = 2$, а требования — $\{a, b\}, \{a, c\}, \{a, d\}, \{e, f\}$. Первые три образуют подсолнух с сердцевиной $\{a\}$. Заменяем их на $\{a\}$ и оставляем $\{e, f\}$. Элементы b, c, d больше не встречаются и удаляются из вселенной. Остаток имеет решения $\{a, e\}$ и $\{a, f\}$; они удовлетворяют и всем исходным требованиям.

Порог леммы — достаточное условие существования, а не запрет искать подсолнух раньше. При $d = 2, k = 2$ общий порог слоя равен $2!\,2^2 = 8$, хотя в нашем примере подсолнух уже виден среди трёх множеств. Для гарантии размера достаточно запускать конструктивную лемму при превышении порога; найденные раньше структуры тоже можно безопасно использовать.

Замена сердцевиной сохраняет допустимые решения до текущего бюджета, но не все большие решения. Это нормально: ядро обязано сохранять ответ заданного экземпляра $(U, \mathcal{F}, k)$, а не отвечать сразу за каждый возможный бюджет.

1.6 Пример и самопроверка

При $k = 2$ множества $\{a, b\}, \{a, c\}, \{a, d\}$ можно заменить на $\{a\}$. Любое решение из двух элементов обязано взять a. При двух таких множествах вместо трёх это неверно: $\{b, c\}$ допу-

стимо. Почему замена уменьшает число требований, но логически может сделать отдельное требование строже?

1.7 Практика

Условия задач 06.1–06.5. Решения открывайте после своей попытки.

1.8 Источники и дополнительное чтение

Письменный источник: Cygan et al., *Parameterized Algorithms*, авторская копия 30.05.2016: §2.6, с. 38–39, теорема 2.25. PDF. Номера страниц печатные; номер страницы PDF больше на 16.

Изложение, поясняющие примеры и упражнения подготовлены для этого курса. Ссылки фиксируют происхождение результатов; для чтения конспекта видео не требуется. Необязательные видео собраны в программе.